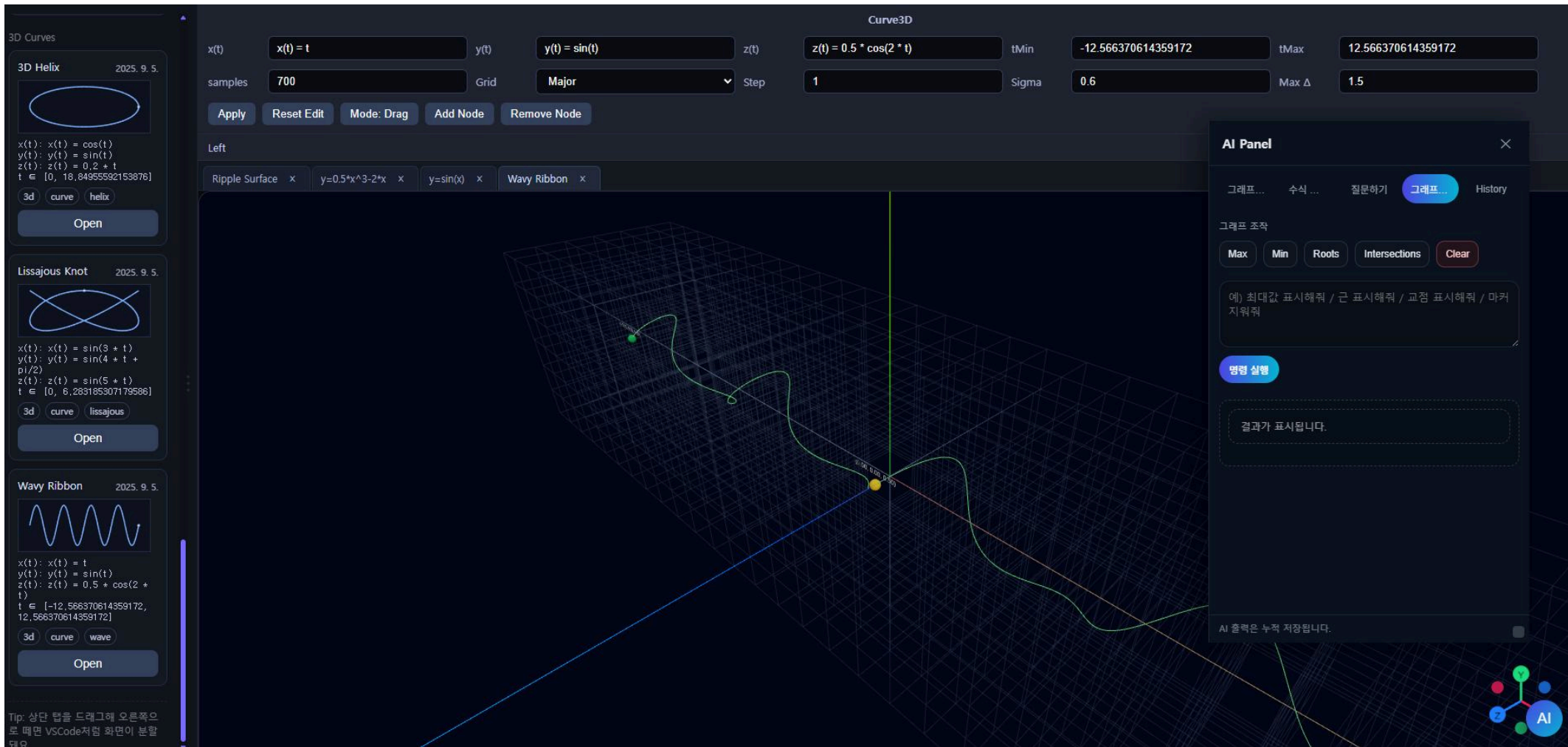




서비스

수식과 데이터를 2D·3D 그래프로 시각화하고, 직접 편집과 AI 분석을 지원하는 인터랙티브 수학 워크스페이스

2D 함수 그래프 3D 매개변수 곡선 3D 곡면

LLM 기반 그래프 분석 및 조작 Vault 기반 리소스 저장·불러오기

[GitHub](#)

기술 스택

 React  JavaScript  Three.js  React Three Fider
 Math.js  Spring Boot  LLM API

역할

Product Design

수식 입력 → 시각화 → 직접 편집 → 저장으로 이어지는 제품 흐름 설계

Graph·Curve3D·Surface3D·Array3D 편집 모드 정의

FrontEnd

Three.js 기반 2D·3D 그래프 렌더링 및 직접 조작 구현

다중 탭·분할 화면·Undo/Redo 구현

AI 명령을 그래프 마커와 분석 결과로 반영

Backend

Spring Boot 기반 인증·Vault·History API 구현

사용자별 수학 리소스 저장·조회·수정 기능 구현

작업 기간

2025. 08 - 2026. 01 (5개월)

구성원

1인 프로젝트

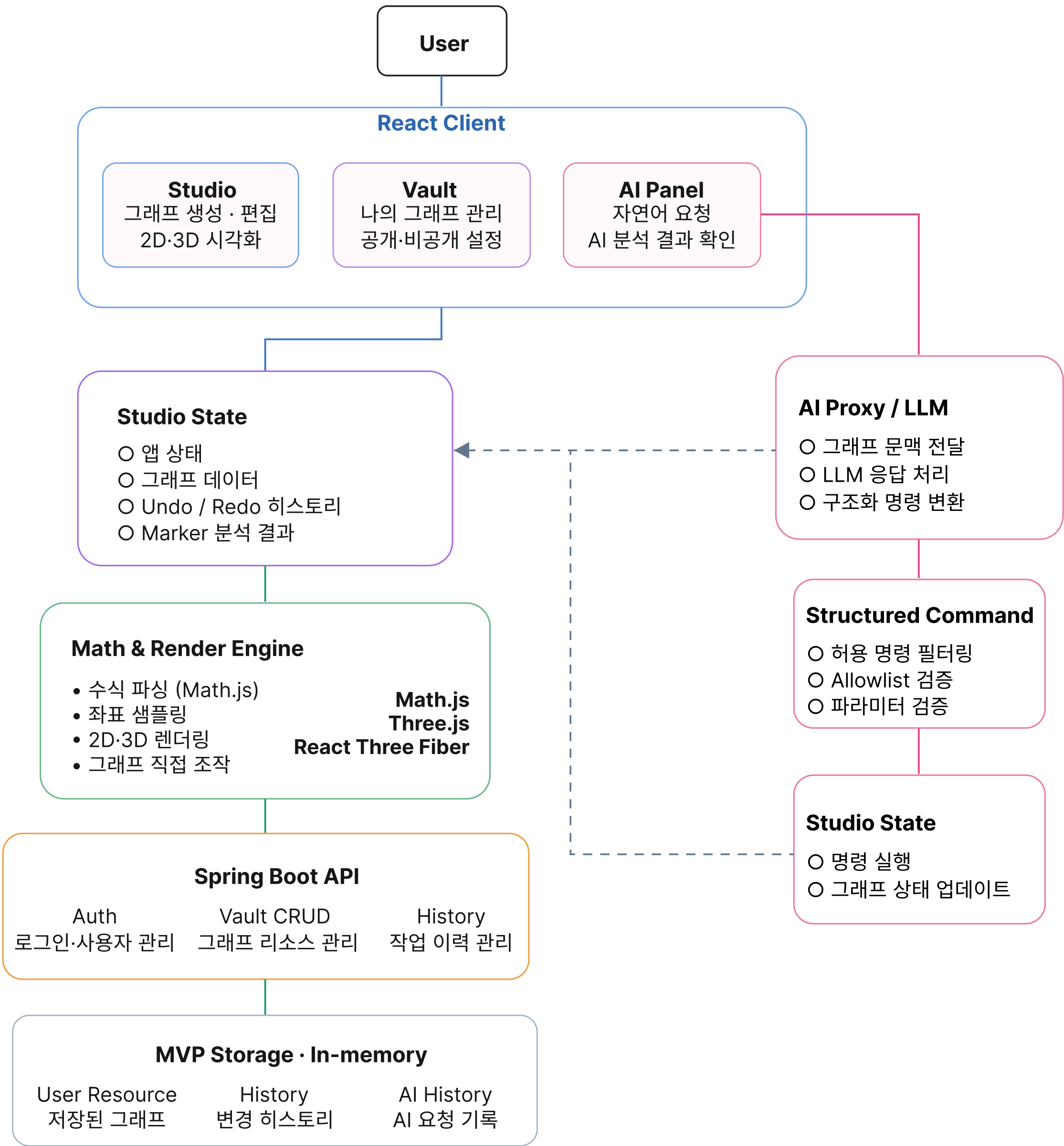
기획 UI/UX 설계 프론트엔드 구현 2D·3D 렌더링

AI 명령 처리 Spring Boot API 구현

프로젝트 회고

- 첫 개인 풀스택 프로젝트
- 정적 그래프 뷰어를 직접 편집 가능한 수학 워크스페이스로 확장
- BufferGeometry 기반 그래픽 데이터 처리 경험 확보
- 기능 확장 과정에서 비대해진 Studio 상태 구조의 모듈화 필요성 확인
- 영속 저장·TypeScript 전환·테스트 강화를 후속 개선 과제로 설정

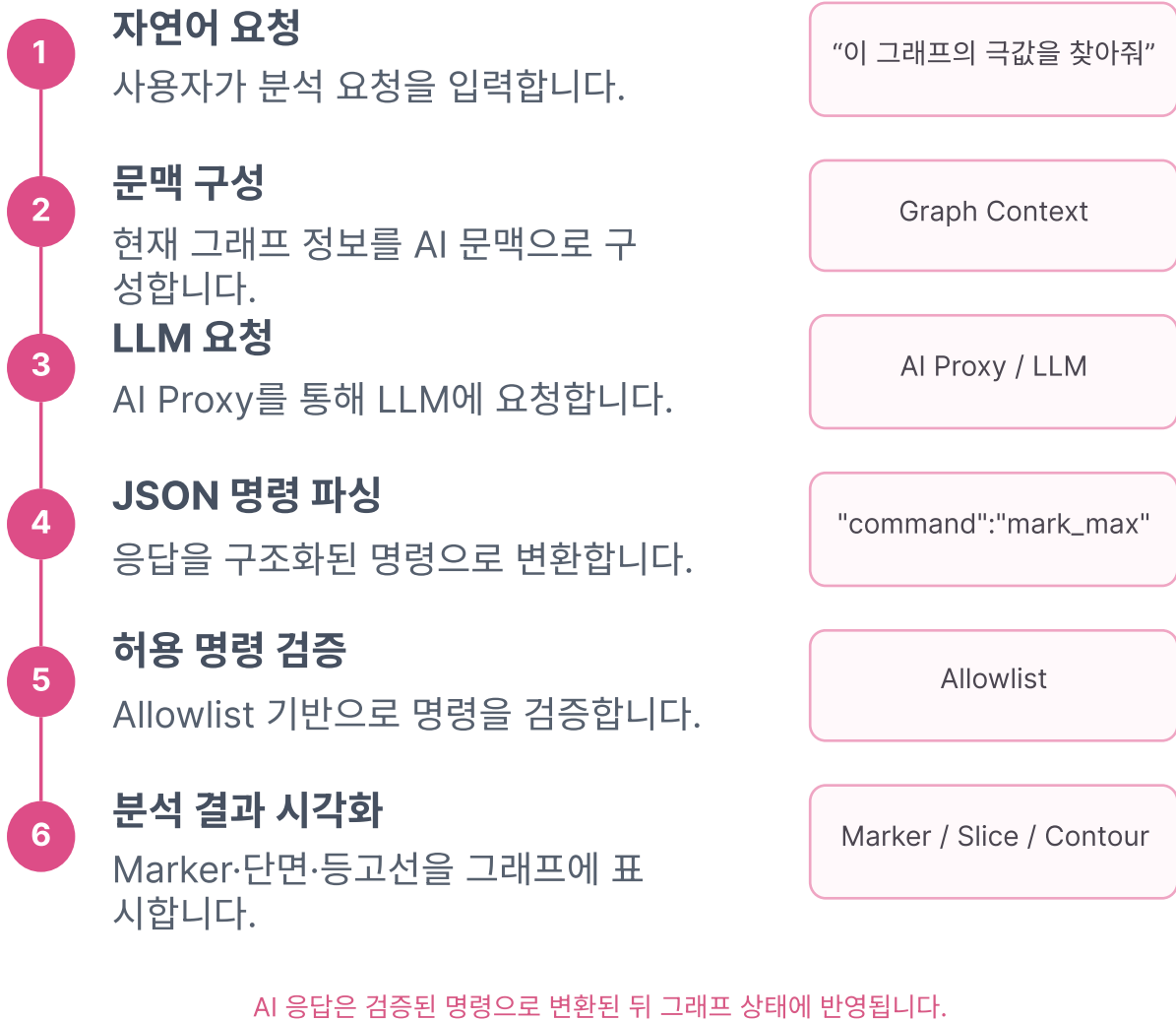
1. 전체 아키텍처



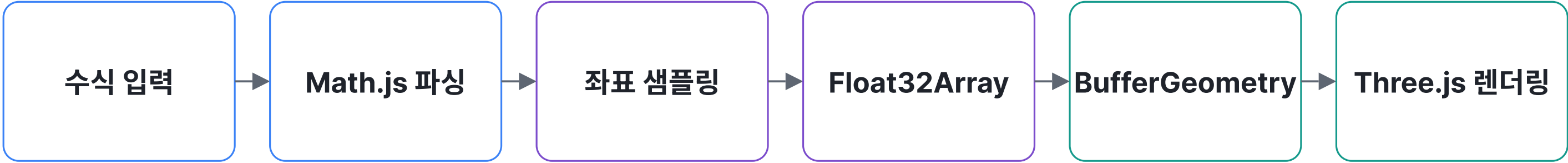
● 그래프 생성 및 편집 흐름



● AI 기반 그래프 분석 흐름



2. 수식 기반 2D·3D 그래프 렌더링



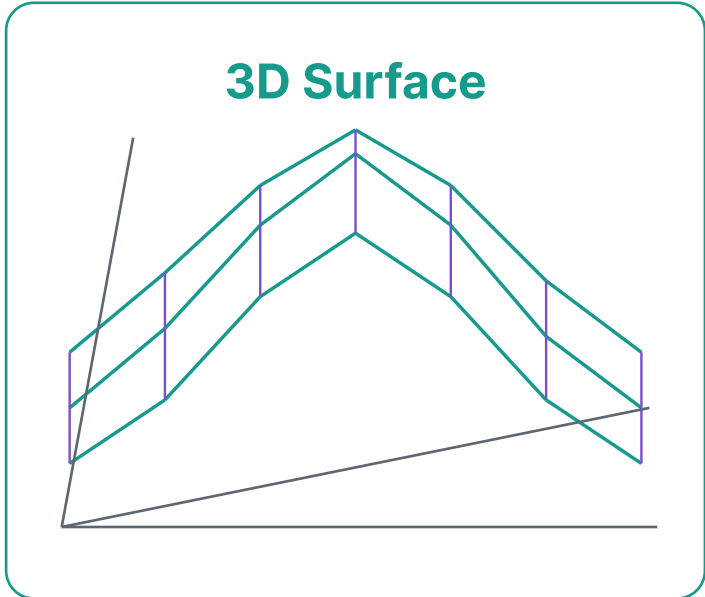
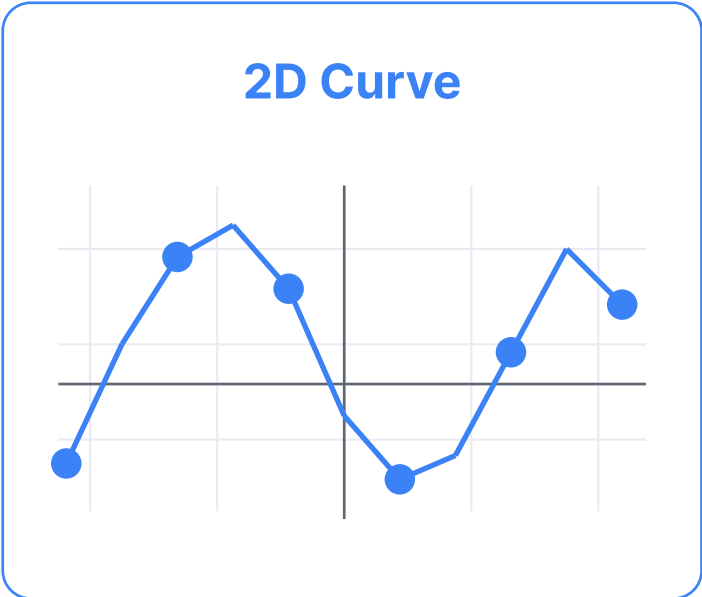
수식을 파싱하고 정의역을 샘플링해 좌표 데이터를 생성한 뒤 BufferGeometry로 변환해 2D·3D 그래프로 렌더링했습니다.

핵심 코드 구조

```
const expr = math.parse(formula);
const compiled = expr.compile();

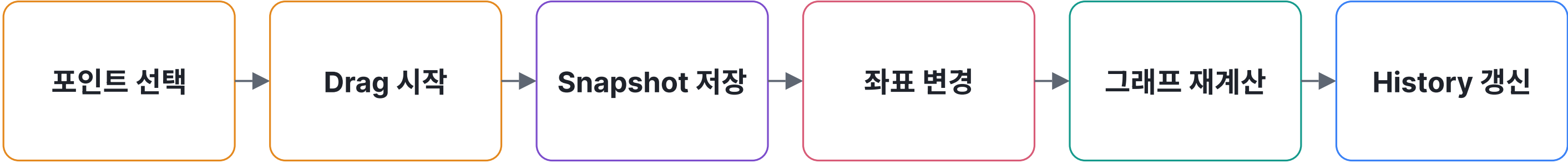
const positions =
  new Float32Array(size);

const geometry =
  new THREE.BufferGeometry();
```

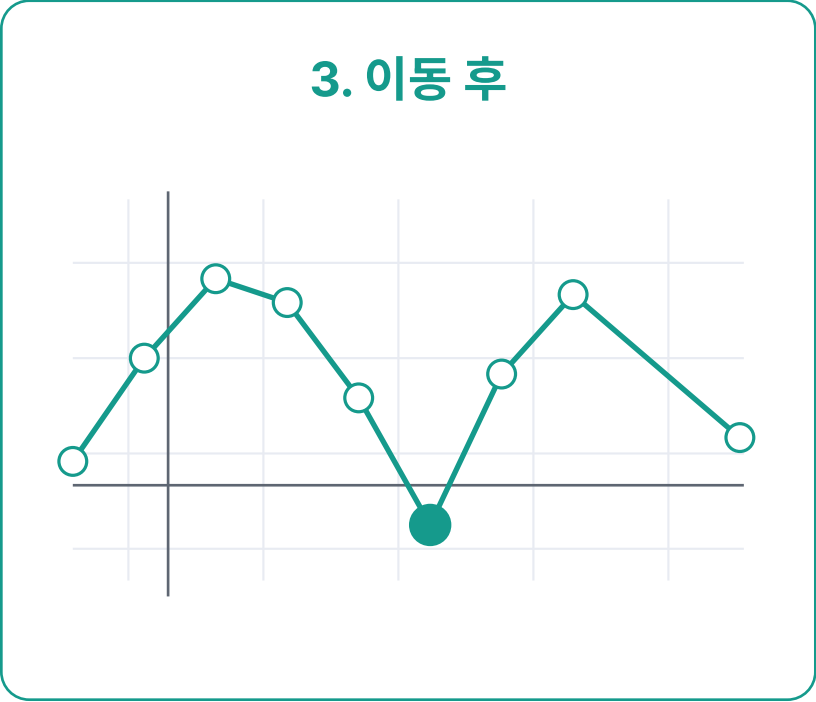
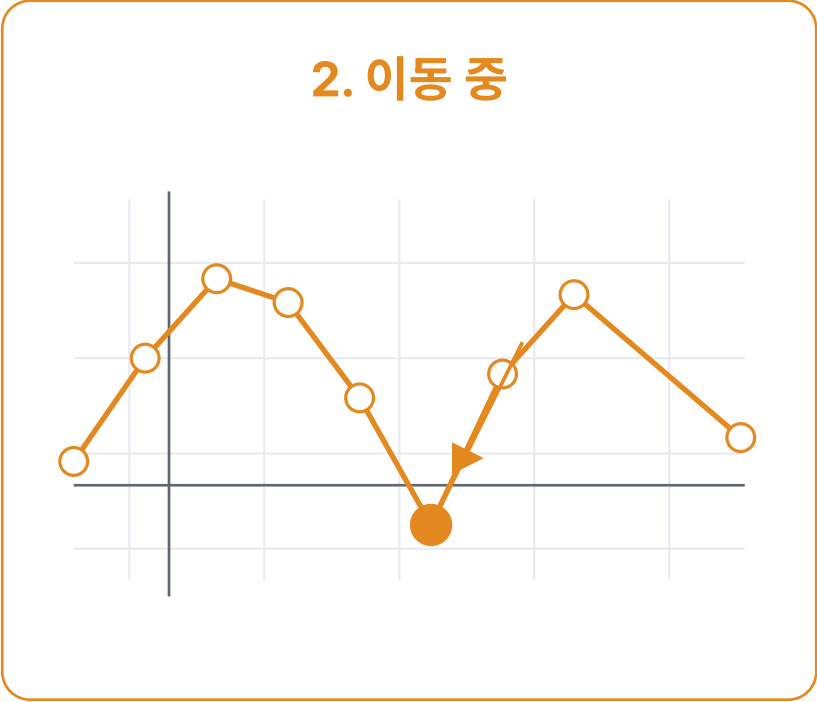
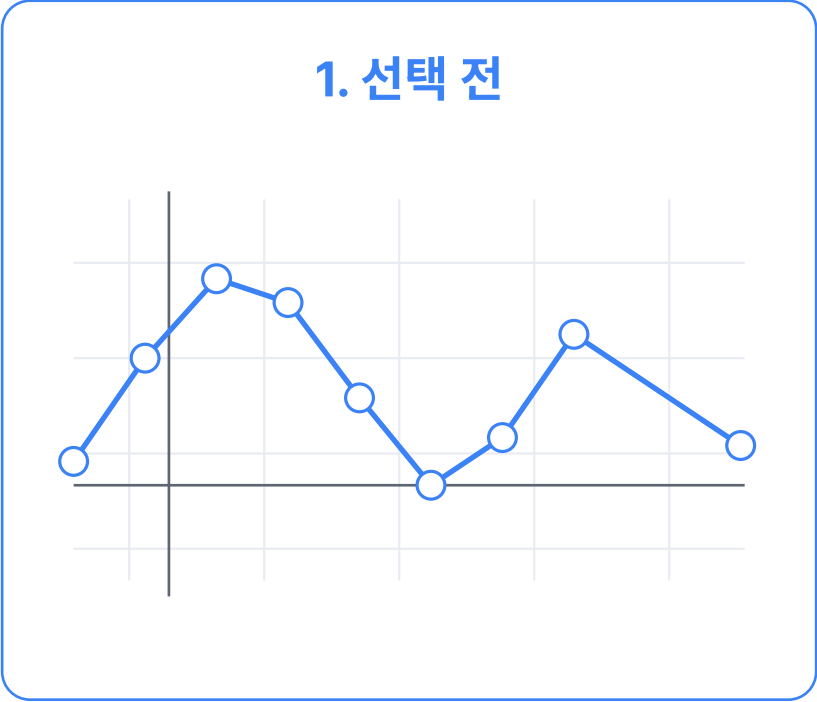


Math.js · Float32Array · BufferGeometry · React Three Fiber

3. 그래프 직접 조작 및 Undo/Redo



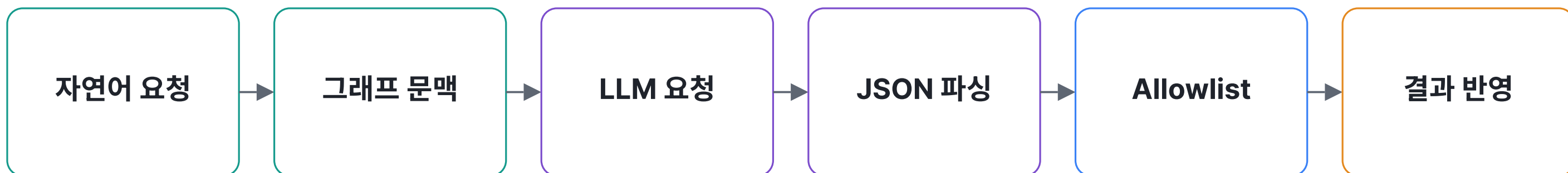
한 번의 Drag 동작을 하나의 Undo 단위로 관리했습니다.



TransformControls · State Snapshot · Undo / Redo

Drag 시작 시 상태 저장 → 종료 시 변경 결과를 History에 기록

4. AI 명령 처리 흐름



그래프 문맥을 LLM에 전달하고, 응답을 JSON 명령으로 변환한 뒤 검증된 결과만 그래프 상태에 반영했습니다.

JSON 명령과 검증 구조

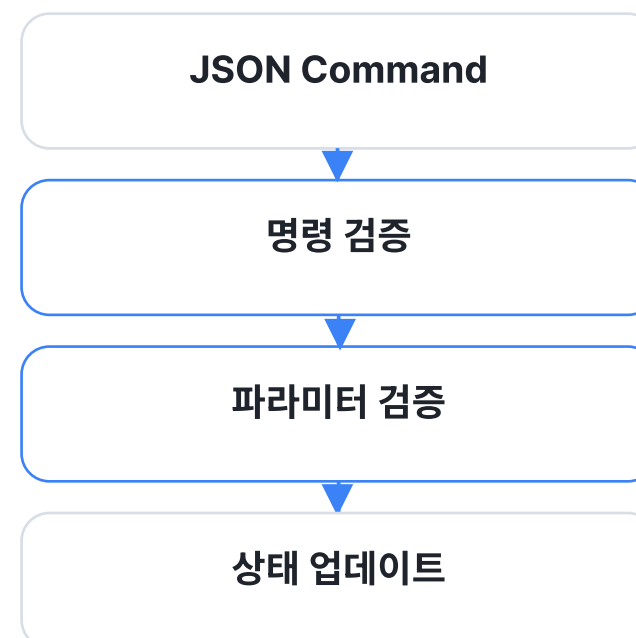
JSON 명령 예시

```
{
  "command": "mark_max",
  "target": "surface",
  "axis": "z",
  "params": {}
}
```

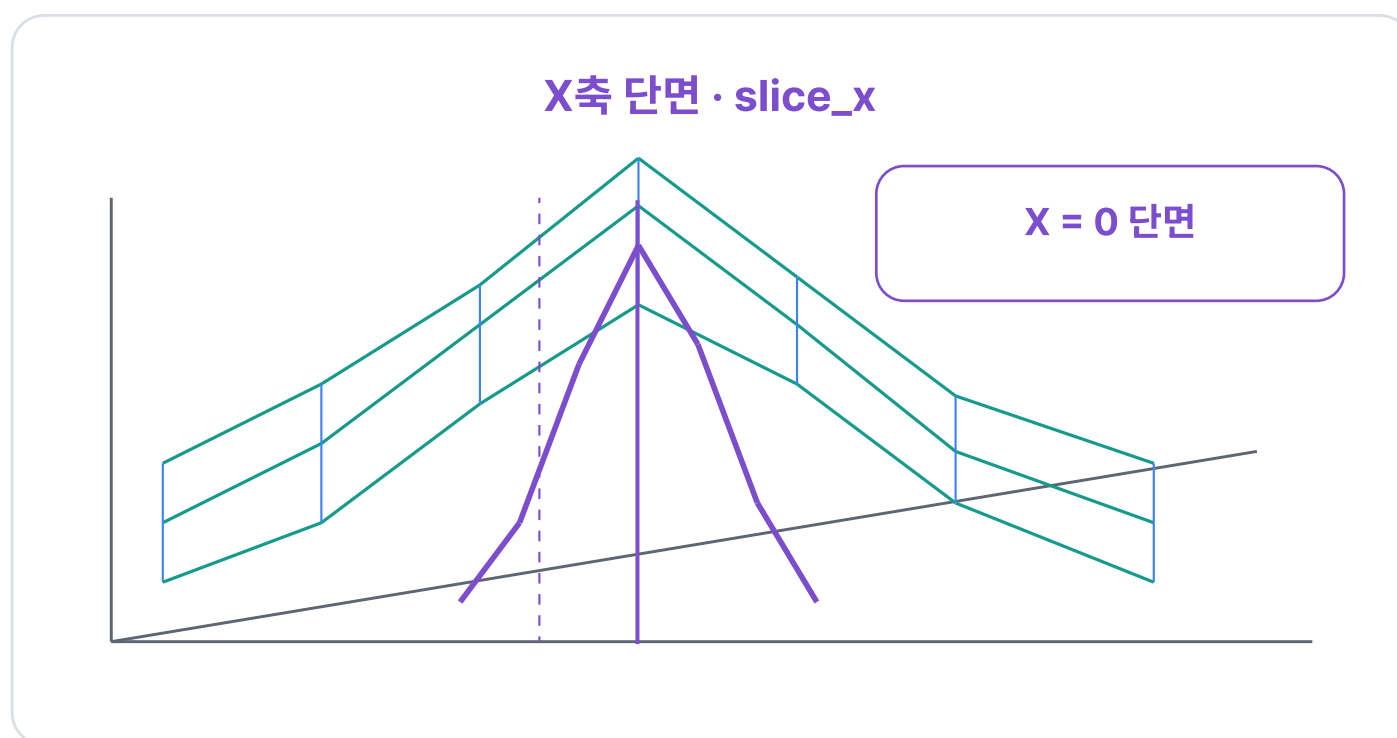
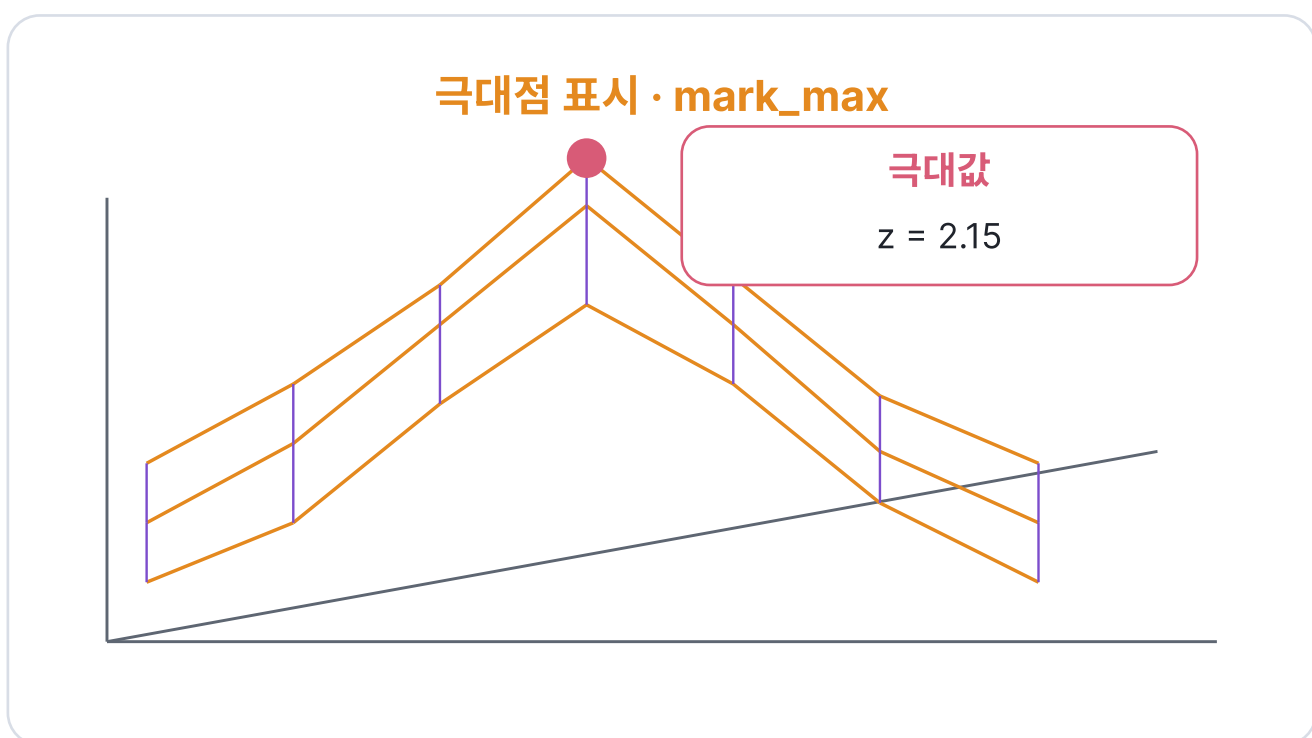
지원 명령 · Allowlist

명령	기능
mark_max	극대점 표시
mark_roots	근 표시
slice_x	X축 단면
contour_z	Z값 등고선

검증 구조



5. 분석 결과 시각화



Context 구성

현재 그래프 정보 포함

명령 검증

JSON + Allowlist

상태 반영

검증된 결과만 적용

문맥 구성 → 명령 파싱 → 검증 → 상태 반영